

Chapter 5

Java I/O Streams

1. Introduction

- Java Input/Output (I/O) Streams provide a mechanism for reading and writing data to various sources and destinations.
- They represent a flow of data, allowing Java programs to interact with files, network connections, memory buffers, and other I/O devices.
- A stream is an abstract sequence of bytes or characters that flows from a source to a destination.
- Streams are unidirectional. There are separate streams for input and output.
- Streams abstract away the underlying details of the I/O device, providing a consistent interface for interacting with different types of data sources and destinations.

1. Introduction

- java.io package provides a rich hierarchy of stream classes.
- Based on the data they handle, stream classes are broadly categorized into Byte Streams and Character Streams.
- Byte Streams:
 - ✓ These streams handle data in the form of raw bytes (8-bit units).
 - ✓ They are primarily used for binary data such as images, audio, video, and compiled code.
- Character Streams:
 - ✓ These streams handle data in the form of characters (16-bit Unicode characters).
 - ✓ They are primarily used for text-based data such as text files, XML, and HTML.
 - ✓ Character streams automatically handle character encoding and decoding.

2. Byte Streams

- InputStream and OutputStream are the core abstract classes for byte streams.

InputStream:

- The abstract superclass for all byte input streams.
- It defines methods for reading bytes of data.
- It's Key methods include:
 - ✓ `int read()`: Reads the next byte of data. Returns the byte as an integer (0-255) or -1 if the end of the stream is reached.
 - ✓ `int read(byte[] b)`: Reads up to `b.length` bytes from the input stream into the byte array `b`. Returns the number of bytes read or -1 if the end of the stream is reached.
 - ✓ `int read(byte[] b, int off, int len)`: Reads up to `len` bytes from the input stream into the byte array `b` starting at offset `off`. Returns the number of bytes read or -1 if the end of the stream is reached.

2. Byte Streams

- ✓ `long skip(long n)`: Skips over and discards `n` bytes of data from the input stream. Returns the actual number of bytes skipped.
- ✓ `int available()`: Returns an estimate of the number of bytes that can be read from this input stream without blocking.
- ✓ `void close()`: Closes the input stream and releases any system resources associated with it.
- ✓ `void mark(int readlimit)`: Marks the current position in the input stream. Subsequent calls to `reset()` will reposition the stream to this marked position. Not all `InputStream` implementations support marking.
- ✓ `void reset()`: Repositions the stream to the last marked position. Throws `IOException` if no mark has been set or the mark is no longer valid.
- ✓ `boolean markSupported()`: Tells whether this input stream supports the `mark()` and `reset()` methods.

2. Byte Streams

OutputStream:

- The abstract superclass for all byte output streams.
- It defines methods for writing bytes of data.
- It's Key methods include:
 - ✓ `void write(int b)`: Writes the specified byte to the output stream. The parameter is an int, but only the lower 8 bits are written.
 - ✓ `void write(byte[] b)`: Writes `b.length` bytes from the specified byte array to the output stream.
 - ✓ `void write(byte[] b, int off, int len)`: Writes `len` bytes from the specified byte array starting at offset `off` to the output stream.

2. Byte Streams

- ✓ `void flush()`: Forces any buffered output bytes to be written to the underlying stream. Some output streams buffer data for efficiency.
- ✓ `void close()`: Closes the output stream and releases any system resources associated with it. It's essential to close output streams to ensure all data is written and resources are freed.

3. Character Streams

- Reader and Writer are the core abstract classes for character streams.

Reader:

- The abstract superclass for all character input streams.
- It provides methods for reading characters.
- Key methods are similar to InputStream but operate on char and String:
 - ✓ `int read()`: Reads a single character. Returns the character as an integer (0-65535) or -1 if the end of the stream is reached.
 - ✓ `int read(char[] cbuf)`: Reads up to `cbuf.length` characters into the character array `cbuf`. Returns the number of characters read or -1 if the end of the stream is reached.
 - ✓ `long skip(long n)`: Skips over and discards `n` characters from the input stream. Returns the actual number of characters skipped.

3. Character Streams

- ✓ `int read(char[] cbuf, int off, int len)`: Reads up to `len` characters into the character array `cbuf` starting at offset `off`. Returns the number of characters read or `-1` if the end of the stream is reached.
- ✓ `boolean ready()`: Tells whether this stream is ready to be read.
- ✓ `void close()`: Closes the reader and releases any system resources associated with it.
- ✓ `void mark(int readAheadLimit)`: Marks the current position in the stream.
- ✓ `void reset()`: Resets the stream to the last marked position.
- ✓ `boolean markSupported()`: Tells whether this stream supports the `mark()` and `reset()` methods.

3. Character Streams

Writer

- The abstract superclass for all character output streams.
- It provides methods for writing characters.
- Key methods are similar to `OutputStream` but operate on `char` and `String`:
- `void write(int c)`: Writes a single character.
- `void write(char[] cbuf)`: Writes the characters in the character array `cbuf`.
- `void write(char[] cbuf, int off, int len)`: Writes `len` characters from the character array `cbuf` starting at offset `off`.
- `void write(String str)`: Writes the characters in the string `str`.

3. Character Streams

- ✓ `void write(String str, int off, int len)`: Writes `len` characters from the string `str` starting at offset `off`.
- ✓ `void flush()`: Flushes the output stream.
- ✓ `void close()`: Closes the writer and releases any system resources.

4. Concrete Stream Classes

- Java provides various concrete implementations of the abstract `InputStream`, `OutputStream`, `Reader`, and `Writer` classes, each designed for specific I/O operations:

Byte Streams:

- File I/O:
 - ✓ `FileInputStream`: Reads bytes from a file.
 - ✓ `FileOutputStream`: Writes bytes to a file.
- Memory I/O:
 - ✓ `ByteArrayInputStream`: Reads bytes from a byte array in memory.
 - ✓ `ByteArrayOutputStream`: Writes bytes to a byte array in memory.

4. Concrete Stream Classes

- Piped I/O (for inter-thread communication):
 - ✓ PipedInputStream: The receiving end of a pipe.
 - ✓ PipedOutputStream: The sending end of a pipe.
- Filtering/Decorator Streams (add functionality to other byte streams):
 - ✓ BufferedInputStream: Adds buffering to an input stream for improved performance.
 - ✓ BufferedOutputStream: Adds buffering to an output stream for improved performance.
 - ✓ DataInputStream: Enables reading primitive Java data types (int, float, boolean, etc.) from an underlying input stream.

4. Concrete Stream Classes

- ✓ `DataOutputStream`: Enables writing primitive Java data types to an underlying output stream.
- ✓ `PrintStream`: Provides convenient methods for printing formatted output (like `System.out` and `System.err`). It automatically flushes and handles exceptions.
- ✓ `ObjectInputStream`: Enables reading Java objects from an underlying input stream (for object serialization).
- ✓ `ObjectOutputStream`: Enables writing Java objects to an underlying output stream (for object serialization).

4. Concrete Stream Classes

Character Streams:

- File I/O:
 - ✓ FileReader: Reads characters from a file using the platform's default character encoding.
 - ✓ FileWriter: Writes characters to a file using the platform's default character encoding.
- Memory I/O:
 - ✓ CharArrayReader: Reads characters from a character array in memory.
 - ✓ CharArrayWriter: Writes characters to a character array in memory.
 - ✓ StringReader: Reads characters from a String.
 - ✓ StringWriter: Writes characters to a String buffer.

4. Concrete Stream Classes

- Piped I/O (for inter-thread communication):
 - ✓ PipedReader: The receiving end of a character pipe.
 - ✓ PipedWriter: The sending end of a character pipe.
- Filtering/Decorator Streams (add functionality to other character streams):
 - ✓ BufferedReader: Adds buffering to a character input stream for improved performance and provides methods for reading lines of text (`readLine()`).
 - ✓ BufferedWriter: Adds buffering to a character output stream for improved performance and provides a method for writing line separators (`newLine()`).
 - ✓ PrintWriter: Similar to `PrintStream` but operates on characters. It provides convenient methods for printing formatted output and can automatically flush.

4. Concrete Stream Classes

- ✓ `InputStreamReader`: Acts as a bridge between byte streams and character streams. It reads bytes from an `InputStream` and decodes them into characters using a specified character encoding (or the platform's default).
- ✓ `OutputStreamWriter`: Acts as a bridge between character streams and byte streams. It encodes characters into bytes using a specified character encoding (or the platform's default) and writes them to an `OutputStream`.
 - The `InputStreamReader` and `OutputStreamWriter` classes are crucial for handling text data when the underlying source or destination is a byte stream (e.g., network connections, compressed files).
 - They allow you to specify the character encoding (like UTF-8, ISO-8859-1, etc.) to ensure correct conversion between bytes and characters.